

地址翻译与虚拟存储硬件

虚拟地址怎样成为可检查、可访问的物理地址

408 · 计算机组成原理 Topic CO-07

母问题：程序地址怎样经过映射与权限检查进入存储层次？

VA → VPN/Offset → TLB / Page Walk → Permission → PA → Cache / Memory

分页只替换页号，保留页内偏移：若 page size 为 2^p ，offset 恒为低 p 位，合法映射下 $PA = (PFN \ll p) | offset$ 。

本册学习范围

- 本册解释程序使用的虚拟地址怎样经过页表和翻译后得到物理地址，并在途中检查有效性、权限和异常。
- 本册重点解释分页、分段、页表项、TLB、多级页表、页内偏移、PIPT/VIPT/VIVT 以及精确重试边界。
- 页框分配、页面置换和缺页后的调度属于操作系统策略；本册只说明硬件检测到什么以及把什么信息交给软件。

1 术语先行：地址翻译的每个缩写都有固定对象

本册的十个关键词

- **VA (虚拟地址)**：程序发出的地址，属于当前进程的地址空间。
- **PA (物理地址)**：主存或设备真正使用的地址。
- **页/页框**：虚拟地址空间和物理内存都按固定大小切分的单位；虚拟页映射到物理页框。
- **VPN/PFN**：虚页号和物理页框号；页内偏移在合法分页映射中保持不变。
- **PTE (页表项)**：记录一页映射到哪里、是否有效以及读写执行权限等信息的内存结构。
- **TLB**：保存近期 VPN 到 PFN 翻译结果的高速缓存；TLB miss 不等于缺页。
- **MMU**：完成地址翻译、权限检查并向 Cache/Memory 发出物理地址请求的硬件单元。
- **Page Walk**：TLB 未命中时，按页表层级逐项读取 PTE 的过程。
- **Page Fault**：页表映射不存在、页面不在主存或权限不允许而产生的同步异常。
- **ASID/PCID**：附在翻译项上的地址空间标识，用来区分不同进程中相同的 VPN。

目录

1 术语先行：地址翻译的每个缩写都有固定对象	1
2 为什么需要间接映射	3
3 PTE 与多级页表	3
4 Page Walk 是依赖链	3
5 分段与段页式：先按逻辑边界，再按固定页映射	3
6 TLB、Fault 与 Cache Miss 三分	4
7 地址空间切换与同步	4
8 TLB 与 Cache 组合	4
9 贯穿母例：一次 Load 的三种“没找到”	4
10 五列概念边界	5
11 做题控制协议	5
12 本册与数据访问的接口	6
13 知识地图：先翻译，再访问数据	6
14 补充细节：虚存是另一层缓存	6
15 PTE 字段、驻留状态与 MMU 责任	7
16 TLB 的组织与位宽推导	7
17 TLB、页表与 Cache 的可行性约束	7
18 段式与段页式的精确算账	8

2 为什么需要间接映射

直接让程序地址等于物理地址会暴露布局、阻碍隔离和共享。分页用固定粒度的 $VPN \rightarrow PFN$ 映射把程序视图与物理布局解耦；页内偏移不参与映射，保证页内寻址结构稳定。

3 PTE 与多级页表

PTE 不是单独的 PFN，通常还包含 valid/present、读写执行权限、特权级、accessed/dirty 或架构扩展位；具体字段和 A/D 位更新方式服从 ISA，不能跨架构硬套。

若 VA 有 v 位、页大小 2^p 、PTE 大小 e ，线性页表大小为

$$2^{v-p} \times e.$$

稀疏地址空间用多级页表只为实际使用的子树分配页。若一页容纳 $N = \text{page size} / \text{PTE size}$ 个 PTE，则每级索引容量为 $\log_2 N$ 位，VPN 按层切分；顶级可能不是完整一层。

4 Page Walk 是依赖链

$$root + index_1 \rightarrow PTE_1 \rightarrow next\ table + index_2 \rightarrow \cdots \rightarrow leaf\ PTE \rightarrow PFN.$$

后一级地址依赖前一级返回内容，不能把 L 级页表机械说成 $L+1$ 次主存：PTE 本身可能在 Cache，访问还受 TLB/存储层次影响。

5 分段与段页式：先按逻辑边界，再按固定页映射

分段把地址写成 $(segment, offset)$ 。段表项给出该段的 base、limit 和保护属性；硬件先检查段号存在、offset 不超过 limit 以及访问权限，再计算

$$PA = base + offset.$$

段的长度可以贴合代码、数据、栈等逻辑单元，便于保护与共享，但段长可变会产生外部碎片，且连续物理空间分配困难。

段页式把一个逻辑地址分成 $(segment, page, offset)$ ：先查段表得到该段的页表基址与界限，检查 offset 所在页不超过段界限；再以 page 查页表得到 PFN，最后保留页内 offset 拼接物理地址。其顺序是

$$segment\ check \rightarrow page\ translation \rightarrow permission \rightarrow PA,$$

不能把段号直接当 VPN，也不能在尚未完成段界限检查时先声称页表访问合法。段页式同时继承段的逻辑保护和分页的固定块分配；代价是额外的表项、检查与翻译缓存组织。

概念 A	≠ 概念 B	真正判据与题面信号	混淆后果
分段	≠ 分页	可变逻辑段 + base/limit vs 固定页 +	把界限检查和页内偏移混写
段页式	≠ 单纯分页	先段检查再页映射，地址多一层逻辑	漏算段表项或错误拼接 PA
段界限失败	≠ 页表无效	逻辑范围/权限错误 vs 页映射不存在	处理者和异常原因写错

6 TLB、Fault 与 Cache Miss 三分

TLB 保存 VPN→PFN 与权限的翻译副本。TLB hit 直接检查权限并拼接 offset；TLB miss 进入 page walk，可能完全找到有效映射，不等于 Page Fault。PTE 无效或权限失败才产生同步异常，后续是否调页、换页或终止由 OS 决定。Cache miss 是数据副本缺失，通常不必进 OS。

事件	缺失/错误对象	快路径	是否必进 OS
TLB miss	翻译缓存项	page walk/refill	不一定
Page Fault	映射无效或不在驻留集	异常交给 OS	是
Permission fault	权限不允许	异常/终止	是
Cache miss	数据块副本	下一级取块	通常否

7 地址空间切换与同步

相同 VPN 在不同进程可映射不同 PFN，TLB 只按 VPN 匹配会产生 homonym。失效、ASID/PCID 等机制区分地址空间。OS 修改 PTE 后，旧翻译副本可能仍被使用，必须按 ISA 规定执行失效和排序；写内存中的 PTE 不等于所有核心立即看到新映射。

8 TLB 与 Cache 组合

PIPT 是 ‘VA→TLB→PA→Cache’，路径直观但串行；VIPT 用 page-offset 位先索引 Cache，同时 TLB 查 VPN，再用 PA tag 比较；若索引侵入 VPN，可能产生 synonym，需额外处理。若 page offset 为 p 、block offset 为 b ，安全索引位至多为 $p - b$ ，每路容量满足

$$sets \times block\ size \leq page\ size.$$

这个约束由页大小、块大小和相联度推出，不背固定容量上限。

9 贯穿母例：一次 Load 的三种“没找到”

1. VPN 不在 TLB：TLB miss，开始 walk；
2. walk 找到有效叶 PTE：refill TLB，形成 PA；
3. PA 在 Cache 中没有 line：Cache miss，向下一级取块；

- 4. walk 找到 invalid/permission fault：硬件报告异常，不能自行从磁盘调页；
- 5. OS 修复映射并完成必要失效后，原指令按 ISA 规则重试；
- 6. 最终 load 数据只在无异常路径提交到目标寄存器。

反例

TLB miss 但 PTE 有效，不是 Page Fault；Cache hit 但权限失败，数据仍不能提交；“缺页后同一指令一定执行两次”也不是稳定模型，应追踪副作用与重试边界。

10 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
VPN	≠ PFN	虚拟页号 vs 物理页框号	错算地址拼接
TLB miss	≠ Page Fault	翻译缓存缺项 vs 映射无效/权限失败	错写处理者
Page Fault	≠ Cache miss	映射/驻留问题 vs 数据副本问题	把硬件路径写成 OS 调页
PTE	≠ TLB entry	内存中的映射结构 vs 翻译缓存副本	忽略失效/同步
VA	≠ PA	程序视图地址 vs 送入物理存储层次的地 址	直接按 VA 做 Cache/内存定位
页框需求	≠ Working Set	正确性底线 vs 动态性能目标	把一次缺页算成颠簸定理

11 做题控制协议

- 1. 由地址宽度和 page size 拆 VPN/offset；
- 2. 由 PTE/page 算每级索引位；
- 3. 画 TLB hit、TLB miss+walk、fault 三支；
- 4. 每级标地址来源、访问对象和权限；
- 5. 得 PA 后再做 Cache 三分；
- 6. 时间问题声明 TLB/Cache 串并行、PTE 是否可缓存；
- 7. 写检测者、处理者、重试点和提交点。

压缩成第一动作

看到虚拟地址题，先问：现在缺的是翻译副本、映射合法性还是数据副本？offset 是否保持不变？

12 本册与数据访问的接口

从虚拟地址到物理访问

本册把程序视图中的 VA 变成带权限和属性的 PA，再把物理地址交给 Cache 或主存。TLB miss、Page Fault 和 Cache miss 的检测者、处理者、重试点与成本必须分开。

地址翻译成本取决于 TLB 命中、page-walk 级数、PTE 是否可缓存、权限检查和与 Cache 的串并行关系；不能把“页表级数”机械等同于固定主存次数，也不能把一次 fault 的 OS/磁盘代价混入普通 Cache miss。

13 知识地图：先翻译，再访问数据

考点	本册机制	接口
分页/页表	VPN/offset、PTE、多级 walk	OS VM
TLB	翻译副本、miss/refill/失效	CO-B02
Cache 组合	PIPT/VIPT、页内偏移约束	CO-06
缺页/置换	只保留 fault 入口与重试边界	OS-04

一句话

翻译先证明“这个地址映射到哪里且有权限”，再把保留 offset 的 PA 交给 Cache/Memory；TLB、Page Fault 和 Cache miss 是三个不同状态机。

本册只讨论硬件可观察的翻译链：拆分地址、查 TLB、走页表、检查权限、形成物理地址并决定是否产生异常。页框分配、置换、工作集、写时复制和缺页修复属于软件策略，除非题设明确给出，否则不能从硬件字段推断。

14 补充细节：虚存是另一层缓存

虚拟存储器同时回答三个问题：程序大于主存时只驻留当前需要的页；多个进程用不同地址空间实现隔离；程序在编译时不必知道最终物理位置。它与 Cache 的共同结构可以这样对照：

维度	Cache ↔ 主存	主存 ↔ 后备存储（虚存）
传送单位	块/Cache line	页
未命中	Cache miss	Page Fault
副本维护者	硬件控制器	硬件检测，OS 处理缺页
映射灵活度	直接/组相联/全相联	通常允许虚页放入任一页框
写回代价	可选 write-through/write-back	后备存储慢，通常以脏页写回

这个类比只用于推导共同结构，不能把 Cache miss 的硬件填充和 Page Fault 的软件修复混为一谈。虚存的高代价解释了为什么映射可以更灵活、并强调缺页后的原指令重试。

15 PTE 字段、驻留状态与 MMU 责任

页表项除了 PFN/页框号，还可能携带下列硬件可见信息；字段名随 ISA 变化，题目给出什么就检查什么：

字段	作用	直接后果
valid/present (装入位)	页是否已有可用物理页框	为 0 进入缺页异常路径
PFN/位置字段	装入时给出页框号，未装入时可记录后备位置	决定翻译结果或交给 OS 的线索
dirty/modified (修改位)	页是否被写过	换出时决定是否需要写回
accessed/reference (访问位)	记录近期是否访问	可供替换策略使用，属于 OS 语义的硬件接口
R/W/X 与特权 级	读、写、执行和访问级别	权限不符产生同步保护异常
cache-disable 等 属性	指定该页的缓存/顺序约束	影响 MMIO 或一致性处理

按装入位和位置字段，可以区分三种状态：已驻留页（valid=1，指向 PFN）、有后备位置但暂未驻留页（valid=0，位置字段可用）以及从未分配的虚页（valid=0 且无有效位置）。硬件只负责检测并报告，选择页框、调入、换出与阻塞由操作系统完成。

MMU 的固定边界

MMU 先拆 VPN/offset，再查 TLB；TLB 未命中时按页表基址和各级索引走 page walk。得到有效 PFN 后，MMU 检查权限并拼接 $PA = PFN \parallel offset$ ，然后才把 PA 交给 Cache/Memory。PTE 无效或权限失败是同步异常；Cache 是否命中不是 MMU 的判断。

16 TLB 的组织与位宽推导

TLB 缓存的是“虚页号 → 物理页框号”的翻译副本，因此页内偏移不进入 TLB 匹配。全相联 TLB 可并行比较所有 VPN；组相联 TLB 则把 VPN 拆为

$$VPN = [\text{TLB tag} \mid \text{TLB set}], \quad \text{sets} = \frac{\text{entries}}{\text{ways}}.$$

若 VA 宽 v 位、页大小为 2^p 字节、TLB 有 E 项且为 w 路，则页内偏移占 p 位，VPN 占 $v-p$ 位，组索引占 $\log_2(E/w)$ 位；TLB tag 为剩余位数。TLB 项中的 PFN 宽度由物理地址宽度决定，而不是由 VA 宽度决定；VA 加宽只会增加 VPN/tag 宽度。命中也要更新替换状态，进程切换或 PTE 修改后必须按 ISA 规定做 ASID/PCID 区分或失效。

17 TLB、页表与 Cache 的可行性约束

把三者各自抽象成 hit/miss 会得到八种组合，但一致性约束会排除其中几种。若 TLB entry 只允许缓存有效映射且换出时同步失效，则“TLB hit + 页表无效”不可能；若 Cache 只缓存主存中的数据，则“页不在主存 + 该 PA 的 Cache hit”也不可能。可行路径应按以下顺序追踪：

TLB	页表/映射	Cache	解释
hit	有效	hit/miss	直接得 PA，再决定数据副本
miss	有效	hit/miss	page walk 得 PA，随后继续 Cache 检查
miss	无效/权限失败	任意	先报告 Page Fault/保护异常，不能把 Cache 命中当成功
hit	无效	任意	若允许出现，说明失效协议被破坏，应先检查题面是否给出陈旧翻译前提

18 段式与段页式的精确算账

段式地址是 $(segment, offset)$ ，段表项给出 base、limit 和保护属性；硬件先检查段号存在、 $offset < limit$ 与权限，再计算 $PA = base + offset$ 。因为段长可变，不能把物理地址写成简单单位拼接。段页式地址是 $(segment, page, offset)$ ：先由段表项取得该段页表基址并完成界限检查，再由段内页号查页表得到 PFN，最后拼接页内偏移。没有 TLB 时，段页式一次数据访问至少包含“查段表、查页表、取数据”三次存储访问；页式为“查页表、取数据”两次，段式为“查段表、取数据”两次。TLB 或页表项可在 Cache 命中时改变实际等待时间，但不改变逻辑依赖链。

陌生虚存题的字段清单

先写 VA/PA 位宽和 page/segment 粒度；再标记每一次访问的对象 (TLB、段表、页表、数据 Cache、主存)；分别记录 valid、权限和 dirty；最后说明异常检测者、软件处理者、返回哪条指令以及是否需要失效/重试。任何“先查 Cache 再证明权限”或“把段表项当 PFN”的解答都在依赖顺序上越界。